Підготовка до заняття 7

Embeddings та векторні бази

r_d

Як читати цей матеріал

Це навчальний текст, а не конспект: він пояснює тему з нуля і спирається лише на заняття 1–6 (токени, виклик моделі через API, вартість запиту, чанки й метадані з заняття 6). На занятті 6 ми перетворили «брудні» документи на чисті чанки з метаданими. Тут — як серед цих чанків швидко знайти ті, що відповідають на питання користувача, навіть якщо він сформулював його зовсім іншими словами.

Усі приклади й цифри — з практики заняття: 69 чанків компанії Northwind Logistics (документи з заняття 6 плюс 20-сторінковий легасі-регламент складу) і 16 питань з відомими правильними відповідями. Наприкінці кожного розділу є блок **«Головне»** і посилання **«Глибше»** (необов'язкові). Розділ 11 — питання для самоперевірки, розділ 12 — середовище і домашня частина практики. Орієнтовний час читання — 60–75 хвилин.

План

1. Навіщо пошук за змістом
лексичний і семантичний пошук, місце в курсі

2. Що таке embedding
текст як вектор, звідки береться «зміст»

3. Vector math без магії
cosine, нормалізація, top-k

4. Як обрати embedding-модель
мова, довжина, ціна, локально чи API

5. Векторні бази та індекси
Qdrant, FAISS, pgvector; HNSW

6. Hybrid search
BM25 і Reciprocal Rank Fusion

7. Reranking
bi-encoder проти cross-encoder

8. Фільтри за метаданими
доступ, версії, pre- і post-filter

9. Embeddings поза пошуком
мультимодальність, дедуплікація, кластери

10. Як міряти якість пошуку
recall@k, MRR, набір питань

11. Перевірте себе
питання для самоперевірки

12. Перед заняттям
середовище і домашня частина практики

Що ви вмітимете після заняття

- генерувати embeddings для свого корпусу і пояснювати, чому пошук видає саме ці результати;
- обирати модель embeddings і векторне сховище під задачу;
- налаштовувати hybrid search, reranking і фільтри за метаданими;
- доводити якість пошуку цифрами (recall@k, MRR), а не враженням.

РОЗДІЛ 1

1 Навіщо пошук за змістом

> Після розділу: ви розумієте, чому класичного пошуку за словами недостатньо для AI-асистента, і де сьогоднішня тема стоїть у курсі.

1.1 Проблема: люди питають не словами документа

На занятті 6 ми отримали чанки: шматки документів з метаданими. Тепер користувач питає: «Скільки днів на тиждень я можу працювати з дому?». Правильна відповідь лежить у чанку політики віддаленої роботи: «Employees who have completed the probation period may work remotely up to three days per week». Спільних слів із запитанням майже немає: «з дому» проти «remotely», «скільки днів» проти «three days per week», ще й мови різні.

Лексичний пошук (Ctrl+F, SQL `LIKE`, пошукові системи на кшталт Elasticsearch) шукає збіги слів. Він чудово знаходить точні терміни, але не розуміє, що «з дому» і «remotely» — про одне. **Семантичний пошук** шукає за змістом: перетворює і питання, і чанки на числа так, що схожий зміст дає схожі числа. Інструмент для цього — **embeddings**.

1.2 Але й семантичного пошуку мало

У практиці є питання «agreement BV-2013-02» — номер угоди з профспілкою, який згадано у виносці регламенту складу. Тут потрібен саме точний збіг коду. Семантичний пошук цей чанк не знаходить узагалі (розділ 10), бо для моделі «BV-2013-02» — майже випадковий набір символів, а лексичний знаходить на першому місці. Тому в продакшні пошук зазвичай **гібридний**: обидва способи разом (розділ 6).

1.3 Місце в курсі



Якість RAG обмежена якістю пошуку: якщо правильного чанка немає серед знайдених, модель або відповість «не знаю», або вигадає. Тому сьогодні ми не лише будуємо пошук, а й міряємо його.

ГОЛОВНЕ

Люди формулюють питання своїми словами, тому пошук за збігом слів пропускає правильні відповіді. **Embeddings** дають пошук за змістом, але гірше знаходять точні коди й назви. Тому пошук для RAG зазвичай **гібридний**, а його якість міряють до того, як будувати відповідь.

Глибше: Pinecone: What is similarity search · Elastic: What is semantic search

РОЗДІЛ 2

2 Що таке embedding

> Після розділу: ви можете пояснити, що таке embedding, звідки в ньому «зміст» і чому чанки й запити мають векторизуватися однією моделлю.

2.1 Текст як точка в просторі

Embedding — це список чисел фіксованої довжини, який модель повертає для тексту. Довжину називають **розмірністю**: у моделі практики — 384 числа, у `text-embedding-3-small` від OpenAI — 1 536. Будь-який текст, від слова до сторінки, стає точкою в цьому багатовимірному просторі.

```
embed("Can I work from home?")
→ [0.021, -0.113, 0.087, ..., 0.042]      # 384 числа
```

Окреме число нічого не означає саме по собі: це не «вимір про відпустки» чи «вимір про ноутбуки». Сенс має лише **взаємне розташування** точок: тексти зі схожим змістом лежать поруч, з різним — далеко.

Аналогія: карта міста

Координати (широта, довгота) самі по собі нічого не кажуть про район. Але якщо дві кав'ярні мають близькі координати, вони поруч, і до однієї легко дійти від іншої. Embedding — така сама «координата змісту», лише вимірів не два, а сотні. Пошук — це «знайди найближчі точки до точки запиту».

2.2 Звідки береться «зміст»

Embedding-модель — це нейромережа, схожа на ті, що ми розбирали на занятті 2, але навчена не генерувати текст, а розставляти тексти в просторі. Її тренують на мільйонах пар «питання — відповідь», «заголовок — стаття», «речення — його переклад»: пари, що пасують одна одній, мають опинитися поруч, а випадкові — далеко. Після такого навчання модель кладе «з дому» поруч із «remotely», бо в навчальних даних вони трапляються в тих самих контекстах.

Звідси два важливі наслідки. По-перше, модель «знає» лише те, що бачила: внутрішній жаргон компанії, коди документів і рідкісні мови вона розставляє гірше. По-друге, багатомовна модель (як у практиці) кладе українське питання поруч з англійським чанком того самого змісту, і пошук працює між мовами.

2.3 Одна модель для чанків і запитів

Кожна модель будує свій простір. Вектор від однієї моделі неможливо порівнювати з вектором від іншої, навіть якщо розмірність однакова: це різні системи координат. Тому:

- чанки й запити векторизує **та сама модель**;
- зміна моделі = **переіндексація всього корпусу**, а не лише нових документів;

- назву моделі зберігають разом з індексом, так само як версію промпту на занятті 4.

ГОЛОВНЕ

Embedding — список чисел, що задає положення тексту в просторі змісту; схожі тексти — поруч. Модель навчена на парах схожих текстів, тому «розуміє» перефраз і навіть інші мови, але гірше — коди й жаргон. **Чанки й запити — однією моделлю**, а зміна моделі означає переіндексацію.

Глибше: OpenAI: Embeddings · Jay Alammar: The Illustrated Word2vec · Sentence Transformers

---

РОЗДІЛ 3

3 Vector math без магії

> Після розділу: ви знаєте, як рахують схожість двох векторів, чому нормалізація важлива і як читати скор пошуку.

3.1 Cosine similarity

Найпоширеніша міра схожості — **косинус кута** між двома векторами. Якщо вектори дивляться в один бік, косинус близький до 1; якщо перпендикулярні (тексти не пов'язані) — близький до 0. Довжина векторів не важить, лише напрям.

```
cos(a, b) = (a · b) / (|a| · |b|)

a · b  = a[0]*b[0] + a[1]*b[1] + ...      # скалярний добуток (dot product)
|a|    = sqrt(a[0]² + a[1]² + ...)         # довжина вектора
```

Приклад у 2D

a = (3, 4), b = (6, 8), c = (4, −3). a · b = 18 + 32 = 50, |a| = 5, |b| = 10 → cos = 50 / 50 = **1**: однаковий напрям, хоча b удвічі довший. a · c = 12 − 12 = 0 → cos = **0**: перпендикулярні, «не пов'язані».

3.2 Нормалізація: пастка з практики

Якщо всі вектори привести до довжини 1 (**нормалізувати**), знаменник дорівнює 1, і cosine стає просто скалярним добутком — це швидше рахувати. Багато моделей (зокрема OpenAI) одразу повертають нормалізовані вектори. Але не всі. Модель у нашій практиці повертає ненормалізовані, і коли ми спершу порахували «схожість» скалярним добутком, отримали значення 8.96 — косинус так не може. Після правильної формули ця сама пара дала 0.66. Qdrant рахує cosine сам, тож пошук не постраждав, а власний код — так.

3.3 Top-k і як читати скор

Семантичний пошук — це «порахувати схожість запиту з кожним чанком і взяти k найближчих» (**top-k**). Результат — список чанків зі скорами. Два правила читання скорів:

- **Скор — не ймовірність.** 0.82 не означає «правильно на 82%». У практиці для українського питання q12 правильний чанк мав скор 0.544 і був лише третім, а два неправильні над ним — 0.563 і 0.553: різниця між «правильно» і «ні» — соті частки.

- **Скори порівнюють лише в межах однієї моделі.** Поріг «релевантно, якщо > 0.8», підібраний для однієї моделі, на іншій не працює. Якщо потрібен поріг, його підбирають на своїх даних.

ГОЛОВНЕ

Схожість векторів міряють **косинусом кута**: 1 — той самий напрям, 0 — не пов'язані. Для нормалізованих векторів cosine = скалярний добуток, але **не всі моделі нормалізують** — перевіряйте. Пошук = top-k найближчих, а скор — відносна міра, не ймовірність і не переноситься між моделями.

Глибше: Wikipedia: Cosine similarity · 3Blue1Brown: Essence of linear algebra

---

РОЗДІЛ 4

4 Як обрати embedding-модель

> Після розділу: ви знаєте, за якими параметрами порівнювати моделі embeddings і скільки коштує векторизувати корпус.

4.1 Параметри, що мають значення

Параметр	Чому важливо
Мова	Англомовна модель погано розставляє українські тексти. Для змішаного корпусу потрібна багатомовна (bge-m3, multilingual-e5, paraphrase-multilingual).
Максимальна довжина	Модель читає лише перші N токенів свого токенізатора, решту мовчки відкидає. У моделі практики N = 128; чанк «4.3.2 Documentation» з таблицею має 832 токени, і більша частина таблиці в embedding не потрапила. Розмір чанка (заняття 6) треба узгоджувати з моделлю.
Розмірність	Більше вимірів — зазвичай трохи точніше, але більше пам'яті й повільніший пошук.
Ціна і де працює	API (OpenAI, Cohere, Voyage) — платите за токени; open-source (bge, e5) — локально, безкоштовно, дані не виходять назовні.
Префікси	Деякі моделі (e5) очікують `query:` і `passage:` перед текстом. Без них якість помітно падає — читайте опис моделі.

4.2 Порівняння популярних моделей

Модель	Вимірів	Ціна	Коли
text-embedding-3-small	1 536	$0.02 / 1 млн токенів	старт через API
text-embedding-3-large	3 072	$0.13 / 1 млн токенів	коли small не вистачає
bge-m3	1 024	$0 + залізо	багатомовні, приватні дані
paraphrase-multilingual-MiniLM (практика)	384	$0, ~220 МБ	швидкий локальний старт

Орієнтир для вибору — лідерборд **MTEB**, але так само, як з LLM на занятті 3, вирішує якість на вашому корпусі й вашій мові. Ціни змінюються, перевіряйте їх перед розрахунком.

4.3 Скільки це коштує

Порахуйте на прикладі

1 млн чанків по 300 токенів = 300 млн токенів × $0.02 = **$6** разово на `text-embedding-3-small`. Корпус кейсу блоку 2 (30–100 документів) — менше цента.

Дорожча частина — пам'ять: 1 млн векторів × 1 536 чисел × 4 байти = **~6 ГБ** RAM для індексу. І переіндексація: змінили модель або chunking — векторизуємо все заново. Тому вектори кешують за хешем тексту (TODO 2 практики): незмінений чанк не векторизуємо вдруге.

4.4 Contextual header

Чанк «| Hotel, per night | 120 EUR | Team lead |» без контексту не каже, що це ліміти витрат. Тому перед векторизацією до тексту додають шлях розділу з метаданих: «Expense Policy > Limits». Це той самий contextual header із заняття 6, і в практиці ми робимо саме так (`text_for_embedding`).

ГОЛОВНЕ

Модель обирають за **мовою**, **максимальною довжиною**, розмірністю, ціною і тим, де вона працює. Векторизувати дешево, дорожче — пам'ять і переіндексація, тому вектори **кешують за хешем**. Чанк довший за ліміт моделі обрізається мовчки, а **contextual header** дає чанку контекст розділу.

Глибше: MTEB Leaderboard · FastEmbed · BAAI/bge-m3

РОЗДІЛ 5

5 Векторні бази та індекси

> Після розділу: ви знаєте, що робить векторна база понад «список векторів», як вона шукає швидко і як обрати сховище.

5.1 Що зберігає векторна база

Для 69 чанків можна просто порахувати схожість з кожним у циклі. Для мільйона — ні: це мільйон множень векторів на кожен запит. **Векторна база** зберігає для кожного чанка три речі: вектор, **payload** (метадані: текст, джерело, розділ, доступ) і **індекс** для швидкого пошуку найближчих. Плюс уміє фільтрувати за payload під час пошуку (розділ 8).

```
from qdrant_client import QdrantClient, models

client = QdrantClient(":memory:")       # у практиці — в пам'яті, без сервера
client.create_collection("northwind", vectors_config=models.VectorParams(
    size=384, distance=models.Distance.COSINE))
client.upsert("northwind", points=[
    models.PointStruct(id=i, vector=vec, payload=chunk) for i, (chunk, vec) in ...])
hits = client.query_points("northwind", query=query_vec, limit=5).points
```

5.2 Як шукати швидко: індекси

Індекс	Як працює	Коли
Flat (exact)	Порівнює запит з кожним вектором. Точно, але час росте лінійно.	До ~100 тисяч векторів
HNSW	Граф «сусідів»: пошук стрибає від далеких точок до близьких, як «знайомий знайомого». Стандарт у Qdrant, pgvector, Chroma.	Більшість продакшн-систем
IVF / PQ	Кластери плюс стиснення векторів.	Сотні мільйонів і більше (FAISS)

HNSW та IVF — це **ANN** (approximate nearest neighbors): заради швидкості вони інколи пропускають справжнього найближчого сусіда. Для курсового проєкту стандартні параметри підходять; важливо лише знати, що пошук наближений, і перевіряти його метриками.

5.3 Яке сховище обрати

Що	Тип	Сильне	Коли
FAISS	бібліотека	дуже швидко, в пам'яті	прототип; немає фільтрів і зберігання метаданих
Chroma	вбудована БД	`pip install` і готово	MVP, курсовий
Qdrant	сервер / хмара / в пам'яті	фільтри за payload, hybrid	від курсового до продакшну
pgvector	розширення Postgres	одна БД для всього	дані вже в Postgres

Обирають за **фільтрами** (без них не працюють права доступу з заняття 6), підтримкою **hybrid** і **операційкою** (хто підтримує, де дані, бекапи). Бенчмарки швидкості на обсягах курсового нічого не вирішують: на десятках тисяч векторів усі відповідають за мілісекунди.

ГОЛОВНЕ

Векторна база = **вектор + payload + індекс + фільтри**. Швидкий пошук дають ANN-індекси (найчастіше **HNSW**), які трохи жертвують точністю. Сховище обирають за фільтрами, підтримкою hybrid і операційкою, а не за бенчмарком.

Глибше: Qdrant: Concepts · Pinecone: HNSW explained · pgvector

---

РОЗДІЛ 6

6 Hybrid search

> Після розділу: ви розумієте, як працює BM25, чому не можна просто скласти скори двох пошуків і як їх об'єднує Reciprocal Rank Fusion.

6.1 BM25: лексичний пошук, зроблений добре

BM25 — класичний алгоритм пошуку за словами, на якому десятиліттями працюють пошукові системи. Він рахує скор чанка за трьома ідеями:

- **Частота слова в чанку**: чим частіше слово запиту трапляється, тим краще, але з насиченням — десятий повтор додає мало.
- **Рідкість слова в корпусі** (IDF): «the» є всюди і майже нічого не важить, «BV-2013-02» є в одному чанку і важить дуже багато.
- **Довжина чанка**: збіг у короткому чанку цінніший, ніж той самий збіг у довгому.

Перед BM25 текст ділять на слова (**токенізація**). У практиці це `re.findall(r"\w+", text.lower())`: «vpn-backup.northwind.example» стає словами vpn, backup, northwind, example. Для мов з багатою морфологією, як українська, слова варто ще зводити до основи (stemming), інакше «посилки» і «посилок» — різні слова.

6.2 Чому не можна просто скласти скори

Для питання «agreement BV-2013-02» BM25 дав правильному чанку скор 13.0, а векторний пошук — найкращому з неправильних 0.36. Шкали різні: BM25 не обмежений зверху, cosine — від −1 до 1. Сума «13.0 + 0.36» нічого не означає, а вага кожного пошуку залежатиме від випадкових особливостей шкали.

6.3 Reciprocal Rank Fusion

RRF ігнорує скори і дивиться лише на позиції (ранги) у кожному списку:

```
score(chunk) = Σ  1 / (60 + rank)      # сума по всіх списках, де є чанк; rank з 1
```

Приклад

Векторний пошук: A, B, C. BM25: C, D. Чанк C: 1/(60+3) + 1/(60+1) = 0.0159 + 0.0164 = **0.0323**. Чанк A: лише 1/(60+1) = 0.0164. C перемагає, бо його знайшли **обидва** пошуки. Константа 60 пом'якшує різницю між першим і п'ятим місцем, щоб один список не домінував.

У практиці векторний пошук пропустив код угоди, BM25 — перефраз і українське питання, а hybrid через RRF знайшов правильний чанк для всіх 16 питань (розділ 10). Qdrant і Weaviate мають hybrid з коробки; ми пишемо RRF самі, щоб побачити, як він працює.

ГОЛОВНЕ

BM25 шукає за словами з вагою рідкісних термінів і знаходить точні коди й назви. Скори BM25 і векторного пошуку в різних шкалах, тому їх не складають, а об'єднують **за рангами через RRF**: 1 / (60 + rank). Hybrid закриває слабкі місця обох способів.

Глибше: Elastic: Practical BM25 · Cormack et al.: Reciprocal Rank Fusion (2009) · Qdrant: Hybrid queries

РОЗДІЛ 7

7 Reranking

> Після розділу: ви розумієте, чому reranker точніший за embeddings, скільки він коштує і де може зашкодити.

7.1 Bi-encoder і cross-encoder

Bi-encoder (embeddings)

Запит і чанк векторизуються **окремо**. Вектори чанків рахуються заздалегідь, тому пошук швидкий навіть на мільйонах. Але весь чанк стиснуто в один вектор ще до того, як модель побачила запит, — частина деталей губиться.

Cross-encoder (reranker)

Модель читає запит і чанк **разом** і оцінює саме цю пару. Точніше, бо бачить, яке слово запиту до якого речення чанка стосується. Але наперед нічого не порахуєш: модель запускається на кожну пару під час запиту.

7.2 Двоетапний пошук

Hybrid
швидко: top-20 кандидатів
→
Reranker
точно: оцінює 20 пар
→
Top-5
у контекст моделі (заняття 8)

Перший етап має не пропустити правильний чанк (високий recall), другий — поставити його нагору (високий MRR). Варіанти reranker: Cohere Rerank і Voyage через API, open-source bge-reranker і ms-marco-MiniLM локально, або LLM як reranker — найдорожче.

7.3 Ціна і пастка з практики

У практиці reranker підняв MRR з 0.82 до 0.88: правильні чанки опинилися вище. Але час на запит зріс з ~3 мс до ~450 мс, а recall@5 упав з 1.00 до 0.94. Причина — **мова**: reranker `ms-marco-MiniLM` навчений лише на англійській і не розуміє українського питання «Скільки днів на тиждень можна працювати віддалено?», тому виштовхнув правильний чанк з top-5, який hybrid знайшов. Для змішаного корпусу потрібен багатомовний reranker (bge-reranker-v2-m3), і рішення «додавати reranker чи ні» приймають за цифрами, а не за замовчуванням.

ГОЛОВНЕ

Reranker (cross-encoder) читає запит і чанк разом і точніше впорядковує результати, але працює лише на невеликій кількості кандидатів і додає latency. Схема: **hybrid → top-20 → reranker → top-5**. Reranker може й зашкодити, якщо не знає мови корпусу, — перевіряйте метриками.

Глибше: SBERT: Cross-encoders · Pinecone: Rerankers · BAAI/bge-reranker-v2-m3

РОЗДІЛ 8

8 Фільтри за метаданими

> Після розділу: ви знаєте, як метадані з заняття 6 обмежують пошук і чому фільтр має працювати під час пошуку, а не після.

8.1 Навіщо фільтри

Поле	Приклад фільтра
`access`	Фінансовий звіт за Q3 бачить лише група finance; регламент складу — лише warehouse.
`effective`	Лише чинні версії: регламент 2016 року з пам'яткою 2017-го, що змінює час відвантаження.
`source`, тип	Шукати лише в інструкціях IT або лише в політиках.
мова	Українські чанки — окремою моделлю чи з окремим reranker.

```
models.Filter(must=[models.FieldCondition(
    key="access", match=models.MatchAny(any=["all_employees"]))])
```

8.2 Pre-filter і post-filter

Post-filter — знайти top-5, а потім викинути заборонене. Якщо всі п'ять заборонені, користувач отримає порожній результат, хоча дозволені релевантні чанки існують. **Pre-filter** — шукати лише серед дозволених. Qdrant і pgvector фільтрують під час пошуку; BM25 у практиці ми фільтруємо самі, перевіряючи `access` кожного кандидата.

Права доступу — у коді пошуку, а не в промпті. Якщо фінансовий чанк потрапив у контекст моделі, жодна інструкція «не показуй це» не гарантує, що модель його не перекаже (заняття 4). У практиці окрема перевірка доводить, що співробітник без групи finance не отримує фінансовий чанк ні з векторного пошуку, ні з BM25.

ГОЛОВНЕ

Метадані з заняття 6 стають **фільтрами пошуку**: доступ, версія, тип, мова. Фільтрують **під час пошуку** (pre-filter), бо post-filter може повернути порожнечу. Доступ контролює код запиту до бази, а не промпт.

Глибше: Qdrant: Filtering · Pinecone: Vector search filtering

РОЗДІЛ 9

9 Embeddings поза пошуком

> Після розділу: ви знаєте, де ще embeddings корисні: інші типи даних, дедуплікація, кластеризація, класифікація.

9.1 Не лише текст

Мультимодальні моделі (CLIP і подібні) кладуть зображення і текст в **один простір**: фото пошкодженої палети опиняється поруч із текстом «damaged pallet», і можна шукати фото за описом. Для аудіо найчастіше роблять транскрипт (Whisper) і далі працюють з текстом. Для сканів — OCR або моделі, що векторизують сторінку як зображення. Для більшості бізнес-задач найпростіше перевести все в текст і мати один індекс.

9.2 Дедуплікація, кластеризація, класифікація

Задача	Як embeddings допомагають
Дедуплікація за змістом	Шинґли з заняття 6 ловлять текстові копії; cosine між embeddings — той самий зміст іншими словами. У практиці (TODO 10) найсхожіша пара з різних документів — «remote work becomes available after probation» і правило віддаленої роботи (0.66): не дубль, а пов'язаний зміст. Поріг дубля підбирають на розмічених парах.
Кластеризація	k-means по векторах звернень клієнтів — і без ручної розмітки видно основні теми.
Класифікація	Найближчі розмічені приклади (k-NN) — класифікатор без навчання моделі.
Semantic chunking	Межа чанка там, де сусідні речення стають несхожими (заняття 6).

ГОЛОВНЕ

Embeddings — універсальний інструмент: мультимодальні моделі шукають зображення за текстом, а в тексті embeddings допомагають у **дедуплікації за змістом**, **кластеризації** і **класифікації** без навчання. Пороги схожості підбирають на своїх даних.

Глибше: OpenAI: CLIP · OpenAI Cookbook: Clustering

РОЗДІЛ 10

10 Як міряти якість пошуку

> Після розділу: ви можете зібрати набір питань для пошуку, порахувати recall@k і MRR і обрати конфігурацію за цифрами.

10.1 Набір питань

Як і golden set на занятті 4, для пошуку потрібні питання з відомими правильними відповідями: для кожного питання — id чанків, що на нього відповідають. У практиці їх 16, і вони навмисно різні: перефраз («My work laptop was stolen on a trip»), точні коди («vpn-backup.northwind.example»), українські питання до англійських документів і питання з обмеженим доступом. Для кейсу блоку 2 достатньо 10–15 питань на вашому корпусі.

10.2 Метрики

Метрика	Як рахуємо	Що показує
recall@k	частка питань, де правильний чанк є в top-k	чи пошук узагалі знаходить відповідь; головна для RAG
MRR	середнє 1 / позиція першого правильного (не знайдено — 0)	наскільки високо стоїть правильний
precision@k	частка правильних серед top-k	скільки шуму піде в контекст
latency	час на запит	чи вкладаємось у бюджет часу

Приклад MRR

Три питання: правильний чанк на 2-му місці, на 1-му, не знайдено. MRR = (1/2 + 1/1 + 0) / 3 = **0.50**. recall@5 = 2 / 3 = **0.67**.

10.3 Результати практики

Конфігурація	recall@5	MRR	мс / запит	Пропущено
vector	0.94	0.76	10	q10 — код угоди BV-2013-02
bm25	0.88	0.69	0	q06 — DPD іншими словами (правильний на 12-му місці); q12 — українське питання
hybrid (RRF)	1.00	0.82	3	—
hybrid + rerank	0.94	0.88	~450	q12 — reranker не знає української

Жодна конфігурація не найкраща за всіма метриками: hybrid знаходить усе, reranker краще впорядковує англійські результати, але повільніший і губить українське питання. Саме так виглядає реальне рішення: таблиця, трейд-оф і вибір під свою задачу. На корпусі з 16 питань одна відповідь — це 6 відсоткових пунктів, тому для серйозних висновків питань потрібно більше.

ГОЛОВНЕ

Якість пошуку доводять цифрами на **наборі питань з правильними чанками**. **recall@k** — чи знайдено, **MRR** — наскільки високо. Конфігурації порівнюють на тих самих питаннях, а вибір — це трейд-оф між recall, порядком і latency.

Глибше: Pinecone: Evaluation measures in IR · Wikipedia: Mean reciprocal rank

РОЗДІЛ 11

11 Перевірте себе

> Спробуйте відповісти без підглядання. Якщо питання викликає складність, поверніться до розділу в дужках.

1. Чому пошук за словами не знаходить відповідь на «Скільки днів можна працювати з дому?», а семантичний знаходить? Де семантичний гірший? *(1.1, 1.2)*
2. Що означає окреме число в embedding? Що має сенс? *(2.1)*
3. Ви змінили модель embeddings лише для нових документів. Що зламається і чому? *(2.3)*
4. Порахуйте cosine для a = (1, 0) і b = (1, 1). *(3.1)*
5. Чому скалярний добуток у практиці дав 8.96, а cosine тієї самої пари — 0.66? *(3.2)*
6. Колега пропонує поріг «релевантно, якщо скор > 0.8», підібраний на іншій моделі. Що не так? *(3.3)*
7. Чанк має 800 токенів, а модель — ліміт 128. Що станеться з embedding і як це виправити? *(4.1)*
8. Що зберігає векторна база, крім векторів, і навіщо індекс HNSW? *(5.1, 5.2)*
9. Чому BM25 знаходить «BV-2013-02», а векторний пошук — ні? *(6.1)*
10. Чому скори BM25 і cosine не складають? Порахуйте RRF для чанка на 2-му місці в одному списку і 1-му в іншому. *(6.2, 6.3)*
11. Чому cross-encoder точніший за bi-encoder, але його не використовують для пошуку по всьому корпусу? *(7.1)*
12. Reranker знизив recall@5 з 1.00 до 0.94. Що сталося і що з цим робити? *(7.3)*
13. Чим небезпечний post-filter для прав доступу? Де має жити перевірка доступу? *(8.2)*
14. Три питання: правильний чанк на 1-му місці, на 3-му і не знайдено. Порахуйте recall@5 і MRR. *(10.2)*

12 Перед заняттям

> На занятті працюємо з кодом з репозиторію курсу. Підготуйте середовище заздалегідь: моделі завантажуються ~300 МБ, на занятті мережа буде перевантажена.

Репозиторій, бібліотеки, моделі

```
git pull                                  # або git clone https://github.com/andbilous/ai-engineeering-2
cd ai-engineeering-2/lesson-07-embeddings
python3.12 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# завантажити моделі заздалегідь (~300 МБ, один раз, у папку .models)
python -c "from fastembed import TextEmbedding; from fastembed.rerank.cross_encoder import TextCrossEncoder; \
TextEmbedding('sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2', cache_dir='.models'); \
TextCrossEncoder('Xenova/ms-marco-MiniLM-L-6-v2', cache_dir='.models')"
```

Ключ API, Docker і Ollama не потрібні: embeddings і reranker працюють локально через `fastembed`, а Qdrant — у пам'яті процесу. Потрібен Python 3.10–3.12: для найновіших версій Python `onnxruntime` може ще не мати збірки.

Файл	Що це
`data/chunks.jsonl`	69 чанків з метаданими: документи заняття 6 і легасі-регламент складу (розділ 1)
`data/questions.json`	16 питань з id правильних чанків, групами доступу і типом (перефраз, код, українською, доступ)
`starter/search.py`	пошук з TODO 1–10
`starter/check.py`	11 перевірок з номером TODO біля кожної

Домашня частина: TODO 1–5

Векторний пошук і метрики робите вдома, а на занятті додаємо BM25, RRF, фільтри доступу і reranker. Пишіть код з AI-асистентом, але кожен рядок маєте вміти пояснити: на занятті хтось із вас покаже свій розв'язок.

Перед кожним TODO в коді є короткий вступ: **мета**, **навіщо** і **результат**, за яким видно, що задачу зроблено. Та сама логіка — в таблицях нижче: якщо слухаєте заняття паралельно з кодом, починайте з колонки «Навіщо».

TODO	Що зробити	Навіщо	Результат
1	`embed()`: векторизувати тексти моделлю `EMBED_MODEL`	пошук порівнює вектори, а не слова (розділ 2)	схожі тексти мають ближчі вектори (перевірка 1)
2	кеш векторів за `sha256(модель + текст)`	модель повільна, незмінене не векторизуємо вдруге (4.3)	другий запуск: `embedded now 0`
3	`build_index()`: колекція Qdrant з cosine, чанк — payload	база швидко знаходить найближчі й повертає чанк з метаданими (5.1)	69 точок з payload (перевірка 3)
4	`vector_search()`: запит тією самою моделлю → top-k	знаходить відповідь, навіть коли слова й мова інші (1.1)	питання про роботу з дому знаходить правило (перевірка 4)
5	`evaluate()`: recall@k і MRR	без цифр не видно, чи hybrid і reranker допомагають (10)	рядок `vector 0.94/0.76` у таблиці

На занятті: TODO 6–10

TODO	Що зробити	Навіщо	Результат
6	BM25 по тих самих чанках	embeddings пропускають точні коди на кшталт BV-2013-02 (6.1)	код угоди — на 1-му місці
7	RRF: об'єднати два списки за рангами	скори в різних шкалах, тому складаємо ранги (6.2–6.3)	hybrid recall@5 = 1.00, вище за vector
8	фільтр доступу в Qdrant і BM25	права з заняття 6 мають працювати в пошуку, а не в промпті (8)	`access filter: OK`
9	reranker для top-20 кандидатів	cross-encoder точніше впорядковує результати (7)	MRR росте, але дивимось на latency і українське питання
10*	пари схожих чанків з різних документів	дублі іншими словами, яких не бачать шинґли (9.2)	`--pairs` друкує 5 пар

* TODO 10 — за бажанням. Запуск:

```
python starter/search.py                       # оцінка всіх конфігурацій
python starter/search.py "vpn backup server"   # результати для одного запиту
python starter/check.py
```

Що побачите

```
FOR CHAT: recall@5/MRR on 16 questions | vector 0.94/0.76 | bm25 0.00/0.00 | hybrid 0.94/0.76
| ...
5/11 checks passed  (vector 0.94 | hybrid 0.94)
```

Без змін у коді скрипт зупиниться з `Not implemented yet`, а `check.py` покаже `0/11`. Після TODO 1–5 — `5/11`: `bm25 0.00` і hybrid, що дорівнює vector, — це нормально, BM25 і RRF робимо на занятті.